Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim:** Simulate cross correlation and autocorrelation on discrete time signals.	
Experiment No: 3	**Date: 27-07-2026**	**Enrollment No: 92400133104**

EXPERIMENT-3

AIM: Simulate cross correlation and autocorrelation on discrete time signals.

Theory:

Cross-correlation and autocorrelation are mathematical operations used to measure the similarity or correlation between two signals. They are widely used in various applications, such as signal processing, image processing, and pattern recognition.

Cross-correlation measures the similarity between two signals at different time shifts. It computes the dot product of one signal with a time-shifted version of the other signal. The resulting cross-correlation signal indicates the similarity between the two signals at different time lags.

Autocorrelation, on the other hand, measures the similarity of a signal with a time-shifted version of itself. It computes the cross-correlation of a signal with itself. The autocorrelation signal shows how the signal is correlated with itself at different time lags.

Flowchart:

The flowchart for the program will consist of the following steps:

Define two discrete-time signals (signal 1 and signal 2) or use the same signal for autocorrelation.
Compute the cross-correlation of the two signals or the autocorrelation of a single signal.
Plot and display the cross-correlation or autocorrelation signal.
Save the cross-correlation or autocorrelation signal (optional).
Now, let's see the Python program that simulates cross-correlation and autocorrelation on discrete-time signals:

Program import
numpy as np
import matplotlib.pyplot as plt

```
def cross_correlation(signal1, signal2):     #
Compute the cross-correlation
   cross_corr = np.correlate(signal1, signal2, mode='full')     return
cross_corr

def autocorrelation(signal):     #
Compute the autocorrelation
```

```
    auto_corr = np.correlate(signal, signal, mode='full')     return
auto_corr

# Define the discrete-time signals signal1 =
np.array([1, 2, 3, 4, 5])
signal2 = np.array([2, 4, 6, 8, 10])

# Compute the cross-correlation
cross_corr = cross_correlation(signal1, signal2)

# Compute the autocorrelation
auto_corr = autocorrelation(signal1)

# Plot the cross-correlation and autocorrelation signals
plt.figure(figsize=(10, 6)) plt.subplot(2, 1, 1)
plt.stem(cross_corr) plt.title('Cross-correlation')
plt.xlabel('Time Lag')
plt.ylabel('Magnitude')

plt.subplot(2, 1, 2) plt.stem(auto_corr)
plt.title('Autocorrelation')
plt.xlabel('Time Lag')
plt.ylabel('Magnitude')

plt.tight_layout()
plt.show()

# Save the cross-correlation or autocorrelation signals (optional)
# np.savetxt('cross_correlation.txt', cross_corr, delimiter=',')
# np.savetxt('autocorrelation.txt', auto_corr, delimiter=',')
```

Explanation:

We import the required modules, including numpy for array operations and matplotlib.pyplot for plotting.
The cross_correlation function computes the cross-correlation between two signals using numpy.correlate with the mode set to 'full'.
The autocorrelation function computes the autocorrelation of a single signal by calling the cross_correlation function with the same signal as both input arguments.
In the main part of the program, we define two discrete-time signals (signal1 and signal2) or use the same signal for autocorrelation.
We compute the cross-correlation by calling the cross_correlation function with signal1 and signal2.
We compute the autocorrelation by calling the autocorrelation function with signal1.

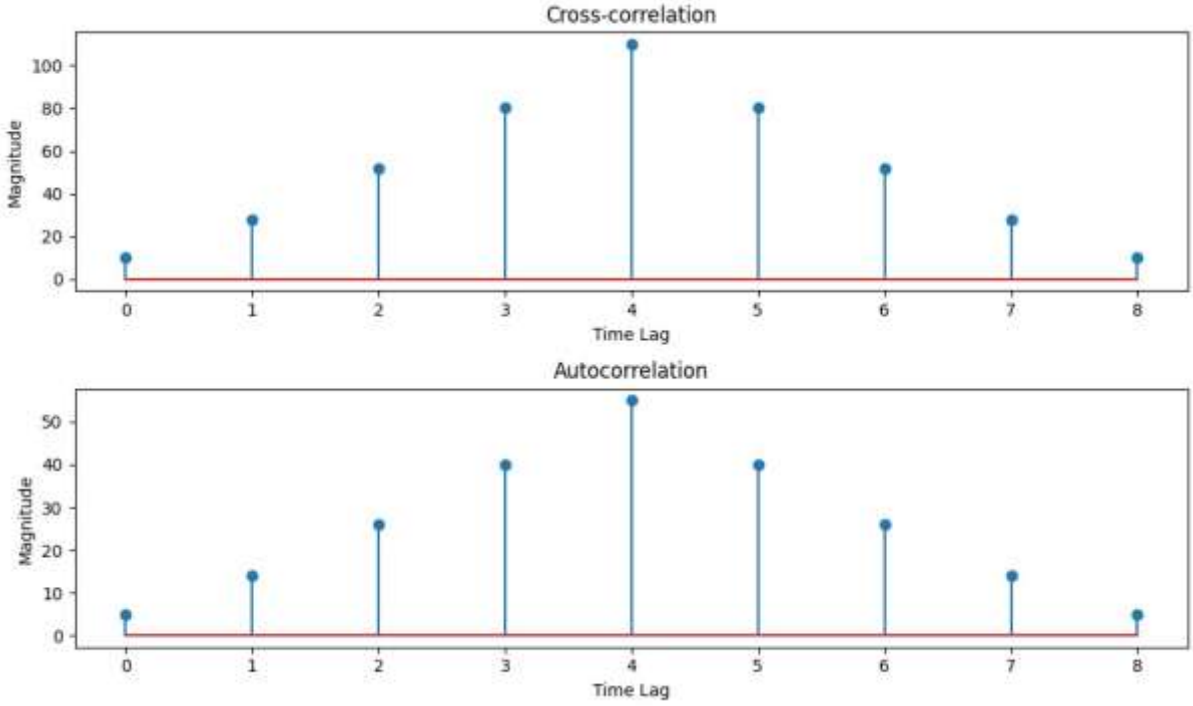
We plot and display the cross-correlation and autocorrelation signals using matplotlib.pyplot.stem.
The cross-correlation and autocorrelation signals can be optionally saved to files using numpy.savetxt.
You can modify the signal1 and signal2 variables to use different discrete-time signals for cross-correlation. Alternatively, you can use the same signal for autocorrelation by assigning signal1 to signal2.

Uncomment the np.savetxt lines if you want to save the cross-correlation or autocorrelation signals to text files.

Make sure you have numpy and matplotlib installed (pip install numpy matplotlib) before running this program.

Expected Output:

A plot will be displayed showing the cross-correlation and autocorrelation signals.
The cross-correlation and autocorrelation signals will be saved to text files (optional). Feel free to experiment with different discrete-time signals and observe the cross-correlation and autocorrelation outputs.



```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import correlate

def load_audio(filename):
    sample_rate, signal = wavfile.read(filename)

    # Convert stereo to mono
    if len(signal.shape) == 2:
        signal = signal.mean(axis=1)

    # Convert to float
    signal = signal.astype(np.float32)

    # Normalize
    signal = signal / np.max(np.abs(signal))

    return sample_rate, signal

def cross_correlation(signal1, signal2):
    return correlate(signal1, signal2, mode='full', method='fft')


def autocorrelation(signal):
    return correlate(signal, signal, mode='full', method='fft')


def normalized_correlation(signal1, signal2):
    corr = np.corrcoef(signal1, signal2)[0, 1]
    return corr


print("Loading audio files...")

sr1, original = load_audio(r"C:\Users\Avni Kavathiya\Desktop\python\.venv\Project Work\Kesariya.wav")

sr2, karaoke = load_audio(r"C:\Users\Avni Kavathiya\Desktop\python\.venv\Project Work\Kesariya Karoake.wav")

sr3, different = load_audio(r"C:\Users\Avni Kavathiya\Desktop\python\.venv\Project Work\wedding song.wav")

print("Audio loaded successfully.")
```

```
duration = 10

original = original[:sr1 * duration]
karaoke = karaoke[:sr2 * duration]
different = different[:sr3 * duration]


original = original[::100]
karaoke = karaoke[::100]
different = different[::100]


min_length = min(len(original), len(karaoke), len(different))

original = original[:min_length]
karaoke = karaoke[:min_length]
different = different[:min_length]


if sr1 != sr2 or sr2 != sr3:
    print("Warning: Sample rates are different.")


print("Computing Cross Correlations...")

corr_original_karaoke = cross_correlation(original, karaoke)

corr_original_different = cross_correlation(original, different)

corr_karaoke_different = cross_correlation(karaoke, different)

print("Cross Correlation Completed.")


print("Computing Auto Correlations...")

auto_original = autocorrelation(original)

auto_karaoke = autocorrelation(karaoke)

auto_different = autocorrelation(different)

print("Auto Correlation Completed.")


coef_ok = normalized_correlation(original, karaoke)

coef_od = normalized_correlation(original, different)
```

```
coef_kd = normalized_correlation(karaoke, different)

print("\n==================================")
print("Normalized Correlation Coefficients")
print("==================================")
print(f"Original vs Karaoke   : {coef_ok:.4f}")
print(f"Original vs Different : {coef_od:.4f}")
print(f"Karaoke vs Different  : {coef_kd:.4f}")

plt.figure(figsize=(12,8))

plt.subplot(3,1,1)
plt.plot(corr_original_karaoke)
plt.title("Cross Correlation: Original vs Karaoke")
plt.xlabel("Lag")
plt.ylabel("Correlation")

plt.subplot(3,1,2)
plt.plot(corr_original_different)
plt.title("Cross Correlation: Original vs Different")
plt.xlabel("Lag")
plt.ylabel("Correlation")

plt.subplot(3,1,3)
plt.plot(corr_karaoke_different)
plt.title("Cross Correlation: Karaoke vs Different")
plt.xlabel("Lag")
plt.ylabel("Correlation")

plt.tight_layout()


plt.figure(figsize=(12,8))

plt.subplot(3,1,1)
plt.plot(auto_original)
plt.title("Autocorrelation - Original Song")
plt.xlabel("Lag")

plt.subplot(3,1,2)
plt.plot(auto_karaoke)
plt.title("Autocorrelation - Karaoke")
plt.xlabel("Lag")

plt.subplot(3,1,3)
plt.plot(auto_different)
plt.title("Autocorrelation - Different Song")
```

```
plt.xlabel("Lag")

plt.tight_layout()

plt.show()
```

